

DRG Cleaning

Carlos Resurreccion

2024-07-01

Load Required Libraries

```
# library(tidyverse)
library(data.table)
library(here)
```

```
## here() starts at C:/Users/resur/Documents/drg-pipeline
```

```
library(tictoc)
```

```
##
## Attaching package: 'tictoc'
```

```
## The following object is masked from 'package:data.table':
##
##      shift
```

```
library(stringr)
library(stringi)
library(lubridate)
```

```
##
## Attaching package: 'lubridate'
```

```
## The following objects are masked from 'package:data.table':
##
##      hour, isoweek, mday, minute, month, quarter, second, wday, week,
##      yday, year
```

```
## The following objects are masked from 'package:base':
##
##      date, intersect, setdiff, union
```

```
library(docstring)
```

```
##
## Attaching package: 'docstring'
```

```
## The following object is masked from 'package:utils':  
##  
##      ?
```

```
library(profvis)  
library(hash)
```

```
## hash-2.2.6.3 provided by Decision Patterns
```

```
##  
## Attaching package: 'hash'
```

```
## The following object is masked from 'package:tictoc':  
##  
##      clear
```

```
## The following object is masked from 'package:data.table':  
##  
##      copy
```

```
# library(foreach)  
# library(doParallel)  
# library(parallel)  
library(future)  
library(future.apply)  
library(knitr)
```

Set Parameters

Year to Load & Version

```
year_to_load <- "2018"  
version <- "v2"
```

Parameters

```
# Parameters  
sample_size <- 25 * 1e3  
seed <- 123  
  
drop_cols <- c(paste0("ICDCODE",13:14),"ICCODED15",paste0("ICDCODE",16:17))  
  
icd_cols <- paste0("clin_icd",1:12)  
rvs_cols <- paste0("clin_rvs", 1:20)  
  
to_read <- FALSE  
to_sample <- TRUE  
to_write <- TRUE
```

```

to_group <- TRUE
to_filter <- FALSE #unused
to_profvis <- FALSE
to_chunk <- FALSE
to_view_checks <- TRUE

# Seed for reproducibility
set.seed(seed)

options(future.globals.maxSize= 1024*1024^2)

global_seed <- seed #for parallelized operations

```

Source Data Formats

```

source(here("data-cleaning", "r_scripts", "data-formats.R"))
source(here("data-cleaning", "r_scripts", "file-paths.R"))

```

Source Functions

```

source(here("data-cleaning", "r_scripts", "general-functions.R"))
source(here("data-cleaning", "r_scripts", "main-functions.R"))
source(here("data-cleaning", "r_scripts", "icd-functions.R"))
source(here("data-cleaning", "r_scripts", "rvs-functions.R"))
source(here("data-cleaning", "r_scripts", "pdx-functions.R"))
source(here("data-cleaning", "r_scripts", "grouper-functions.R"))

```

Load Mapping Data

```

proc <- fread(here(path_to_excel, "proc.csv"))
proc[, CODE := as.character(CODE)]

rvs_icd9 <- fread(here(path_to_aux, "rvs_icd9cm.csv"),
                  select = c("rvs", "icd9cm"))
rvs_icd9[, rvs := as.character(rvs)]
rvs_icd9[, icd9cm := as.character(icd9cm * 100)]
rvs_icd9 <- merge(rvs_icd9, proc[, .(CODE, DRGUSE)],
                  by.x = "icd9cm", by.y = "CODE", all.x = TRUE)
rvs_icd9[, is_drg := !is.na(DRGUSE) & DRGUSE]
rvs_icd9 <- rvs_icd9[!is.na(rvs) & !is.na(icd9cm), -"DRGUSE"]

acr_rvs <- fread(here(path_to_aux, "acr_rvs.csv"))

# Read in the data.table
tdrg_icd10 <- fread(here(path_to_aux, "i10.csv"))

# Set the key if not already set

```

```

setkey(tdrg_icd10, "CODE")

# Subset and assign the result to acc_pdx
acc_pdx <- tdrg_icd10[ACCPDX == "Y", CODE]

# Optional: if CODEs are not unique in tdrg_icd10
acc_pdx <- unique(acc_pdx)

```

Read Data

Reading Data

```

dt <- main_read_function()

## [1] "Sampled file exists. Reading the sampled file..."
## [1] "Sampled file matches sample size."

if (file.exists(total_rows_file)) {
  total_rows <- readRDS(total_rows_file)
} else {
  total_rows <- fread(full_claims, select = 1L, header = TRUE)[, .N]
  saveRDS(total_rows, file = total_rows_file)
}

```

Data Processing

Data Cleaning

```

if (!to_chunk){
  tic("Total execution time:")
  if (to_profvis) {
    profvis({
      dt <- clean_data(dt)
    })
  } else {
    dt <- clean_data(dt)
  }
}

```

Not chunking

## [1] "id_series"	"id_pin"	"date_adm"
## [4] "time_adm"	"date_dis"	"time_dis"
## [7] "date_rec"	"date_ref"	"date_check"
## [10] "id_hci"	"id_hcp"	"clin_outpatient"
## [13] "clin_emergency"	"pat_type"	"clin_acc"
## [16] "pat_rel"	"pat_bdate"	"pat_age"

```

## [19] "pat_sex"          "pat_bwt"          "pat_memcat_parent"
## [22] "pat_memcat_child" "pat_memcat_subchild" "clin_discharge"
## [25] "clin_c1"          "clin_c2"          "clin_icd1"
## [28] "clin_icd2"        "clin_icd3"        "clin_icd4"
## [31] "clin_icd5"        "clin_icd6"        "clin_icd7"
## [34] "clin_icd8"        "clin_icd9"        "clin_icd10"
## [37] "clin_icd11"       "clin_icd12"       "clin_rvs1"
## [40] "clin_rvs2"        "clin_rvs3"        "clin_rvs4"
## [43] "clin_rvs5"        "clin_rvs6"        "clin_rvs7"
## [46] "clin_rvs8"        "clin_rvs9"        "clin_rvs10"
## [49] "clin_rvs11"       "clin_rvs12"       "clin_rvs13"
## [52] "clin_rvs14"       "clin_rvs15"       "clin_rvs16"
## [55] "clin_rvs17"       "clin_rvs18"       "clin_rvs19"
## [58] "clin_rvs20"       "claim_status"     "claim_payout"
## [61] "claim_charge"     "date_ext"         "id_year"
## [1] "Successfully renamed columns; All expected columns exist"
##      clin_c1_orig clin_c1
##      <char> <char>
## 1:      A97.1    A971
## 2:      I21.9    I219
## 3:      K29.1    K291
## 4:      J06.9    J069
## 5:      N39.0    N390
## 6:      P23.9    P239
##      clin_c2_orig clin_c2
##      <char> <char>
## 1:      E86.1    E861
## 2:      I63.4    I634
## 3:      I61.5    I615
## 4:      E86.1    E861
## 5:      I63.9    I639
## 6:      E87.8    E878
## [1] "No RVS codes discarded"
## [1] "No RVS codes discarded"
##
##
## |Column          | "" Replaced| "NA" Replaced| "character(0)" Replaced|
## |:-----:|-----:|-----:|-----:|
## |id_series       |          0|          0|          0|
## |id_pin          |          0|          0|          0|
## |date_adm        |          0|          0|          0|
## |time_adm        |          2|          0|          0|
## |date_dis        |          0|          0|          0|
## |time_dis        |          2|          0|          0|
## |date_rec        |          2|          0|          0|
## |date_ref        |       22145|          0|          0|
## |date_check      |       1734|          0|          0|
## |id_hci          |          0|          0|          0|
## |id_hcp          |         33|          0|          0|
## |pat_type        |          0|          0|          0|
## |clin_acc        |         908|          0|          0|
## |pat_rel         |      15363|          0|          0|
## |pat_bdate       |       5471|          0|          0|
## |pat_sex         |          0|          0|          0|

```

```
## |pat_memcat_parent |          3|          0|          0|
## |pat_memcat_child  |          3|          0|          0|
## |pat_memcat_subchild|          3|          0|          0|
## |clin_discharge    |         16|          0|          0|
## |clin_c1           |       11936|          0|       293|
## |clin_c2           |         703|          0|       293|
## |claim_status      |          0|          0|          0|
## |date_ext          |          0|          0|          0|
## |clin_icd          |          0|          0|      8970|
## |clin_rvs          |          0|          0|          0|
## [1] "Patient Types:"
## [1] "DEP" "MEM"

## Warning in remap_memcat_parent_desc(pat_memcat_parent): Unmapped member
## category parents found: NA

## [1] "Memcat Parent Types:"
## [1] "INDIRECT" "DIRECT" NA

## Warning in remap_memcat_child_desc(pat_memcat_child): Unmapped member category
## children found: NA

## [1] "Memcat Child Types:"
## [1] "INDIGENT" "SENIOR" "INFORMAL" "FORMAL" "LIFETIME" "SPONSORED"
## [7] NA

## Warning in remap_disposition(clin_discharge): Unmapped clinical discharge
## dispositions found: NA

## [1] "Discharge Types:"
## [1] 1 2 9 4 NA 3
```

```
if (to_chunk){
  tic("Total execution time:")
  if (to_profvis) {
    profvis({
      num_cores <- max(1, availableCores() - 1)
      chunk_size <- ceiling(nrow(dt) / num_cores)
      chunks <- split(dt, rep(1:num_cores, each = chunk_size,
                             length.out = nrow(dt)))

      # Plan for parallel processing
      plan(multisession, workers = num_cores)
      # Process each chunk in parallel
      processed_chunks <- future_lapply(chunks, process_chunk,
                                         future.seed = global_seed)

      # Combine processed chunks
      dt <- rbindlist(processed_chunks)
      dt <- replace_empty_with_na(dt, to_view_checks)
    })
  }
}
```

```

} else {
  num_cores <- max(1, availableCores() - 1)
  chunk_size <- ceiling(nrow(dt) / num_cores)
  chunks <- split(dt, rep(1:num_cores, each = chunk_size,
                           length.out = nrow(dt)))

  # Plan for parallel processing
  plan(multisession, workers = num_cores)
  # Process each chunk in parallel
  processed_chunks <- future_lapply(chunks, process_chunk,
                                    future.seed = global_seed)

  # Combine processed chunks
  dt <- rbindlist(processed_chunks)
  dt <- replace_empty_with_na(dt, to_view_checks)
}
}

```

Chunking

Map codes and Find PDx (if not chunking)

```

#Map RVS codes
if (!to_chunk){
  if (to_profvis) {
    profvis({
      dt <- map_rvs_icd9(dt, rvs_icd9)
      dt <- implement_icd10_mapping(dt)
      # Replace empty strings in character and factor columns with NA
      # dt <- replace_empty_with_na(dt, to_view_checks)
    })
  } else {
    dt <- map_rvs_icd9(dt, rvs_icd9)
    dt <- implement_icd10_mapping(dt)
    # Replace empty strings in character and factor columns with NA
    # dt <- replace_empty_with_na(dt, to_view_checks)
  }
}

```

Map Codes

```

## There are 3323 RVS codes without an ICD-9CM equivalent recognized by the TDRG ICD9CM
## Of these, 744 (100.00%) have a mapping to an ICD-9-CM code.
## Of these, there are 162 (21.77%) with more than one ICD9 equivalent recognized by the Thai ICD9 library
## There are 0 (0.00%) with no ICD-9-CM equivalents.
## There are 2761 unique entries for ICD-10 codes, of which 2110 (76.42%) are directly in the Thai ICD-10 library
## The modifications led to a total of 2503 codes being mapped to an equivalent in the Thai ICD10 library
## Out of these, 392 were modified to match.
## There are 258 codes that could not be mapped to the Thai ICD10 library:
## Index: <code>
##       code  source
##       <char> <char>

```

```
## 1: NSD01 clin_c1
## 2: MCP01 clin_c1
## 3: A971 clin_icd
## 4: FP001 clin_c2
## 5: ANC01 clin_c1
## ---
## 254: X59 clin_icd
## 255: J922 clin_icd
## 256: M9330/3 clin_icd
## 257: J162 clin_icd
## 258: V99 clin_icd
```

```
#Map RVS codes
if (!to_chunk) {
  if (to_profvis) {
    profvis({
      # dt <- apply_find_pdx(dt)
      pdx_result <- apply_find_pdx(dt$clin_c1, dt$clin_c2, dt$clin_icd)
      dt$pdx <- pdx_result$pdx
      dt$pdx_code <- pdx_result$pdx_code
    })
  } else {
    # dt <- apply_find_pdx(dt)
    pdx_result <- apply_find_pdx(dt$clin_c1, dt$clin_c2, dt$clin_icd)
    dt$pdx <- pdx_result$pdx
    dt$pdx_code <- pdx_result$pdx_code
  }
}
```

```
if (to_write) {
  fwrite(dt, here(path_to_intermediate, paste0("output_", year_to_load,
                                                suffix, ".csv")))
}
```

Find PDx

Export

Export for Batch Grouper

```
if (to_group) {
  if (to_profvis) {
    profvis({
      export_for_batch_grouper(dt, year_to_load, output_txt_file)
      for_batch_grouping <- fread(output_txt_file,
                                  sep = "|", na.strings = "--")
      batch_grouping_result <- fread(grouper_result_file,
```



```

        sep = "|", na.strings = "--")
    })
  } else {
    export_for_batch_grouper(dt, year_to_load, output_txt_file)
    for_batch_grouping <- fread(output_txt_file,
                                sep = "|", na.strings = "--")
    batch_grouping_result <- fread(grouper_result_file,
                                   sep = "|", na.strings = "--")
  }
}

```

Runtime Estimation

Stop Timer

```

# Stop the timer and capture total time
toc_data <- toc(log = TRUE)

```

```
## Total execution time:: 18.43 sec elapsed
```

```
total_time <- toc_data$toc - toc_data$tic
```

Calculate Speed

```

# Calculate time spent per cell and per row
total_rows_dt <- nrow(dt)
total_cells <- nrow(dt) * ncol(dt)

time_per_cell <- total_time / total_cells
time_per_row <- total_time / total_rows_dt
time_estimate_total_rows <- time_per_row * total_rows

# Format the row numbers
formatted_total_rows_dt <- format_large_numbers(total_rows_dt)
formatted_total_rows <- format_large_numbers(total_rows)

# Print the results with aligned decimal points and formatted row numbers
cat(sprintf("Time spent (total) for %2s rows: %1.2f sec (actual)\n",
            formatted_total_rows_dt, total_time))

```

```
## Time spent (total) for 25.0k rows: 18.43 sec (actual)
```

```

cat(sprintf("Time spent (t/row) for %2s rows: %1.2f msec (actual)\n",
            formatted_total_rows_dt, time_per_row * 1000))

```

```
## Time spent (t/row) for 25.0k rows: 0.74 msec (actual)
```

```
cat(sprintf("Time spent (total) for %2s rows: %2.2f min (estimate)\n",  
           formatted_total_rows, time_estimate_total_rows / 60))
```

```
## Time spent (total) for 11.8m rows: 144.71 min (estimate)
```